

# Basic Java Guide

A beginner-friendly guide to understanding Java fundamentals.

---

## Table of Contents

1. Introduction to Java
  2. Installing Java
  3. Your First Java Program
  4. Java Syntax Basics
  5. Variables and Data Types
  6. Operators
  7. Conditional Statements
  8. Loops
  9. Methods
  10. Arrays
  11. Object-Oriented Programming (OOP)
  12. Constructors
  13. Inheritance
  14. Exception Handling
  15. Collections
  16. File Handling
  17. JDBC Database Connection
  18. Best Practices
  19. Conclusion
- 

## Introduction to Java

Java is a:

- High-level programming language
- Object-Oriented language
- Platform-independent language

## Features of Java

- Write Once, Run Anywhere (WORA)
  - Secure
  - Robust
  - Multi-threaded
  - Portable
  - Easy to learn
-

# Installing Java

## Step 1 — Install JDK

Download JDK from:

- Oracle JDK
- OpenJDK

## Step 2 — Verify Installation

```
java -version  
javac -version
```

---

# Your First Java Program

## Hello World Example

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

## Output

```
Hello World
```

---

# Java Syntax Basics

## Important Rules

- Every statement ends with `;`
  - Java is case-sensitive
  - Class names start with uppercase
  - Method names start with lowercase
-

# Variables and Data Types

## Primitive Data Types

| Type    | Example |
|---------|---------|
| int     | 10      |
| double  | 10.5    |
| char    | 'A'     |
| boolean | true    |
| long    | 100000L |

## Example

```
public class Variables {  
  
    public static void main(String[] args) {  
  
        int age = 25;  
        double salary = 50000.50;  
        char grade = 'A';  
        boolean active = true;  
  
        System.out.println(age);  
        System.out.println(salary);  
        System.out.println(grade);  
        System.out.println(active);  
    }  
}
```

# Operators

## Arithmetic Operators

| Operator | Example |
|----------|---------|
| +        | a + b   |
| -        | a - b   |
| *        | a * b   |
| /        | a / b   |

| Operator | Example |
|----------|---------|
| %        | a % b   |

## Example

```
int a = 10;
int b = 5;

System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
```

## Conditional Statements

### If Statement

```
int age = 20;

if(age >= 18) {
    System.out.println("Adult");
}
```

### If-Else Statement

```
int marks = 40;

if(marks >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

### Switch Statement

```
int day = 2;

switch(day) {
    case 1:
        System.out.println("Monday");
}
```

```
        break;

    case 2:
        System.out.println("Tuesday");
        break;

    default:
        System.out.println("Invalid Day");
}
```

---

## Loops

### For Loop

```
for(int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

---

### While Loop

```
int i = 1;

while(i <= 5) {
    System.out.println(i);
    i++;
}
```

---

### Do-While Loop

```
int i = 1;

do {
    System.out.println(i);
    i++;
} while(i <= 5);
```

---

# Methods

Methods are reusable blocks of code.

## Example

```
public class Calculator {  
  
    static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
  
        int result = add(10, 20);  
  
        System.out.println(result);  
    }  
}
```

---

# Arrays

Arrays store multiple values.

## Example

```
public class ArrayExample {  
  
    public static void main(String[] args) {  
  
        int[] numbers = {10, 20, 30, 40};  
  
        for(int num : numbers) {  
            System.out.println(num);  
        }  
    }  
}
```

---

# Object-Oriented Programming (OOP)

## Main OOP Concepts

1. Class
  2. Object
  3. Encapsulation
  4. Inheritance
  5. Polymorphism
  6. Abstraction
- 

## Classes and Objects

### Example

```
class Car {  
  
    String brand;  
    int speed;  
  
    void drive() {  
        System.out.println("Car is moving");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Car car = new Car();  
  
        car.brand = "Toyota";  
        car.speed = 180;  
  
        car.drive();  
    }  
}
```

---

## Constructors

Constructors initialize objects.

## Example

```
class Student {  
  
    String name;  
  
    Student(String studentName) {  
        name = studentName;  
    }  
}
```

## Inheritance

Inheritance allows one class to inherit another.

## Example

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

## Exception Handling

Used to handle runtime errors.

## Try-Catch Example

```
public class ExceptionExample {  
  
    public static void main(String[] args) {  
  
        try {
```



```
        int result = 10 / 0;
    } catch (Exception e) {
        System.out.println("Error occurred");
    }
}
}
```

---

## Collections

Java Collections Framework stores groups of objects.

### ArrayList Example

```
import java.util.ArrayList;

public class ListExample {

    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<>();

        names.add("John");
        names.add("Mary");

        System.out.println(names);
    }
}
```

---

## File Handling

### Writing to a File

```
import java.io.FileWriter;

public class FileExample {

    public static void main(String[] args) {

        try {
            FileWriter writer = new FileWriter("test.txt");
            writer.write("Hello Java");
            writer.close();
        }
    }
}
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

---

## JDBC Database Connection

### Example

```
import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

    public static void main(String[] args) {

        try {

            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb",
                "root",
                "password"
            );

            System.out.println("Connected Successfully");

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

---

## Best Practices

### Coding Standards

- Use meaningful variable names
- Keep methods small
- Use comments wisely
- Handle exceptions properly
- Close resources

## Performance Tips

- Avoid unnecessary objects
  - Use StringBuilder for large strings
  - Use collections efficiently
- 

## Common Java Tools

| Tool          | Purpose               |
|---------------|-----------------------|
| IntelliJ IDEA | IDE                   |
| Eclipse       | IDE                   |
| Maven         | Dependency Management |
| Gradle        | Build Tool            |
| Spring Boot   | Backend Framework     |
| MySQL         | Database              |

---

## Conclusion

Java is one of the most powerful and widely used programming languages.

Learning Java fundamentals helps you build:

- Backend Applications
- Enterprise Systems
- Android Apps
- APIs
- Microservices
- Desktop Applications

Master the basics first before moving to:

- Spring Boot
- Microservices
- Docker
- Kubernetes
- Cloud Development